

Описание Python-скрипта Book Scanner

Версия описания соответствует текущей архитектуре скрипта v12.9: автоматическое извлечение страниц книги из видео, выпрямление перспективы, deskew текста, удаление пальцев, фильтрация дублей и финальная постобработка JPEG.

Назначение	Автоматически получить чистые JPEG-страницы из видео перелистывания книги
Вход	MOV/MP4 или другой видеофайл, читаемый OpenCV
Выход	page_001.jpg, page_002.jpg ... плюс debug CSV при --debug
Ключевая идея	Сначала стабильно выбрать лучшие кадры, затем улучшать только выбранные страницы

1. Используемые библиотеки

OpenCV / cv2	Чтение видео, поиск контуров, маски, перспективное преобразование, поворот, фильтры, морфология, inpainting, резкость.
NumPy	Работа с массивами пикселей, геометрией точек, медианами, процентилями, проекциями по строкам и столбцам.
MediaPipe, опционально	Если установлен, помогает находить руки. Если его нет, скрипт использует OpenCV fallback по цветовым признакам кожи.
argparse, csv, pathlib, dataclasses	Интерфейс командной строки, debug-таблицы, пути, структуры данных и служебная логика.

2. Общий pipeline

видео → выбор кадров → поиск страницы → перспективный warp → deskew → оценка качества → удаление дублей → финальная чистка → улучшение изображения → JPEG

- Скрипт берёт кадры не из каждого FPS, а с заданной частотой, например --sample-fps 3.
- Для каждого кадра ищется область страницы, похожая на светлую бумагу.
- Каждая найденная страница превращается из трапеции в прямоугольник.
- Дальше считается качество: резкость, текст, руки, перекрытия, площадь и похожесть на другие страницы.
- После выбора победителей запускается финальная постобработка, которая уже не влияет на выбор кадров.

3. Поиск страницы

Страница ищется через маску «похоже на бумагу». Для этого изображение переводится в цветовые пространства HSV, LAB и YCrCb. Пиксели считаются частью страницы, если они достаточно светлые, не слишком насыщенные и похожи на бумажный фон.

После пороговой сегментации применяются морфологические операции: dilate, erode, open/close. Они закрывают дырки в маске, убирают мелкий шум и помогают получить большую связную область страницы.

Контуры ищутся через cv2.findContours, затем приближаются к четырёхугольнику через cv2.approxPolyDP или резервные методы на основе bounding box / minAreaRect.

4. Перспективное выпрямление

Главная геометрическая операция — гомография. Если страница в кадре выглядит как трапеция, скрипт находит четыре угла и строит матрицу 3×3, которая переводит эту трапецию в прямоугольник.

```
cv2.getPerspectiveTransform(src_points, dst_points)
cv2.warpPerspective(image, matrix, output_size)
```

В однородных координатах это записывается как $[x', y', w']^T = H \cdot [x, y, 1]^T$, после чего реальные координаты равны $X = x'/w'$ и $Y = y'/w'$.

Размер результата считается по евклидовым расстояниям между углами: ширина — максимум верхней и нижней стороны, высота — максимум левой и правой стороны. Затем длинная сторона масштабируется до значения --long-side, например 1800 пикселей.

5. Deskew: выравнивание текста

После perspective warp страница может оставаться слегка повернутой. Скрипт ищет тёмные компоненты, похожие на буквы, группирует их в строки и оценивает угол базовой линии текста.

Для поворота используется аффинное преобразование cv2.getRotationMatrix2D и cv2.warpAffine. Математически это обычный поворот: $x' = x \cos \theta - y \sin \theta$, $y' = x \sin \theta + y \cos \theta$.

В v12.9 добавлен финальный refined deskew: он применяется только к уже выбранным страницам, чтобы не менять выбор кадров. Он лучше учитывает короткие титульные строки и крупные заголовки.

6. Оценка качества кадра

Резкость	Variance of Laplacian: cv2.Laplacian(gray).var(). Чем выше значение, тем резче кадр.
Текстовая плотность	Доля тёмных пикселей/компонент, похожих на текст. Помогает отсеивать пустые и нечитаемые кадры.
Площадь страницы	Страница должна занимать достаточно большую часть кадра и иметь правдоподобное соотношение сторон.
Руки/пальцы	Оценивается размер и положение маски руки, особенно перекрытие текста и нижней части страницы.
Похожесть	dHash/perceptual hash помогает понять, являются ли две страницы дублями.

7. Детекция пальцев и inpainting

Если доступен MediaPipe, скрипт использует готовую модель руки и landmarks. Если MediaPipe не установлен, работает fallback: поиск кожных пикселей по HSV/YCrCb/LAB, затем проверка формы и размера маски.

```
cv2.inpaint(image, mask, radius, cv2.INPAINT_TELEA)
```

В поздних версиях добавлены строгие проверки: маска не должна быть слишком большой, слишком светлой или похожей на саму бумагу. Это защищает от серых пятен на пустых страницах.

8. Удаление дублей и выбор страниц

Одна страница обычно попадает в несколько кадров. Скрипт выбирает локальные пики качества, группирует близкие по времени и визуально похожие кадры, а затем оставляет лучший кандидат: более резкий, менее закрытый рукой и более чистый.

Если задано --expected-pages 5, алгоритм старается получить ровно 5 уникальных страниц, но всё равно отбрасывает мусорные кадры и дубли.

9. Логика page-side и auto-smart

В v12.9 режим по умолчанию — `--page-side auto-smart`. Он сначала выбирает победившие кадры без рискованной обрезки, а затем повторно обрабатывает только эти страницы и аккуратно убирает внутренний корешок/сливер, если это безопасно.

Перед обрезкой проверяется уверенность seam/корешка, доля обрезки, визуальная похожесть до/после через dHash similarity и максимальная допустимая потеря ширины. Поэтому smart-обрезка не меняет количество страниц и не меняет выбранные кадры.

auto-smart	Режим по умолчанию. Стабильный выбор кадров плюс безопасная финальная smart-обрезка края.
auto	Старое консервативное поведение без smart-обрезки. Полезно для fallback.
right / left	Принудительно брать правую или левую сторону разворота.
full	Сохранять весь найденный четырёхугольник без деления на стороны.

10. Финальная чистка

Нижняя тёмная полоса

`trim_bottom_dark_strip` анализирует яркость нижних строк. Если они значительно темнее тела страницы, отрезается небольшой край, по умолчанию не больше 5% высоты.

Фон и контраст

`enhance_scanned_page` выбирает режим обработки: `standard` для обычных страниц, `sparse` для почти пустых страниц, `decorative` для страниц с рисунками и крупной графикой.

Sparse

Очищает почти пустые страницы от серой пятнистости: отключает CLAHE, мягко выравнивает фон и сглаживает бумажные вариации.

Decorative

Защищает страницы с рисунками/обложечным дизайном от агрессивного отбеливания и постеризации.

11. Debug-вывод

При `--debug` скрипт сохраняет `scores.csv` и `winners.csv`. `scores.csv` содержит оценки всех кандидатов, `winners.csv` — финальные выбранные страницы и диагностику постобработки.

```
page, frame_idx, t_sec, raw_score, norm_score, peak_score,
hand, bottom_hand, blur, text, page_side_mode,
trim_applied, seam_confidence, shrink, dhash_similarity,
bottom_trim_px, cleanup_applied, enhance_mode, deskew_angle_final
```

12. Ключевая математика

- Пороговая сегментация по цвету и яркости.
- Морфология бинарных масок: `erode`, `dilate`, `open`, `close`.

- Контуры и аппроксимация многоугольников.
- Евклидова геометрия точек и расстояний.
- Гомография 3×3 для perspective transform.
- Аффинный поворот для deskew.
- Variance of Laplacian для резкости.
- Connected components для текста, графики и масок.
- Проекция яркости по строкам и столбцам.
- Медианы и проценты для устойчивых оценок.
- Perceptual hashing / dHash для схожести страниц.
- TELEA inpainting для удаления пальцев.

13. Команда запуска

```
python extract_book_pages_v12_9.py IMG_4883.MOV \
--expected-pages 5 \
--sample-fps 3 \
--debug \
--long-side 1800
```

14. Главная архитектурная идея

Самая важная особенность текущей версии: потенциально рискованные улучшения выполняются после выбора страниц. Сначала алгоритм стабильно решает, какие кадры являются лучшими страницами, и только затем применяет smart-обрезку, очистку нижнего края, inpainting, улучшение фона и refined deskew.

Это уменьшает риск регрессий: улучшение одной страницы не должно менять набор выбранных страниц или порядок результата.